School of Science
Engineering and Technology

COSC-2753: Machine Learning

Assignment 2 – 2025A

Lecturer: Bao Nguyen Thien

Team member:
Huynh Ngoc Tai (s3978680)
Tran Quang Minh (s3988776)
Kim Nhat Anh (s3978831)
Dang Cuu Dang Khoa (s3979159)

Submission Due Date: 16/05/2025

"I declare that in submitting all work for this assessment I have read, understood and agree to the content and expectations of the Assessment declaration

I. Problem Overview

This project aims to predict wheat plant characteristics - health, variety, and age - using image-based machine learning models. It focuses on evaluating model performance and potential applications in precision agriculture.

II. Preparing for the project

1. Dataset Description and Structure: We were received a train_images.zip containing wheat plant images organized by disease label (with "normal" for healthy plants) and a meta_train.csv file mapping each image ID to its disease label, variety, and age. Additionally, test_images.zip contains unlabeled images for evaluation, with predictions submitted via prediction_submission.csv.

2. Project Environment Setup: In this assignment, we use TensorFlow and Keras as our main frameworks for developing image-based neural networks. All model training is performed locally on our own machine, rather than using web-based platforms like Google Colab or Kaggle. To maintain a clear workflow, each task is handled in a separate file for better control and organization.

III. Preprocessing Data

1. Initial Data Preparation

We began by extracting the contents of the provided zipped dataset. Using the meta_train.csv file, we matched each image ID with its corresponding disease label, variety, and age. To facilitate easier access and further processing, we also mapped each image ID to its file path and added this information to the dataset. This structured preparation ensures efficient navigation and analysis in the subsequent steps.

2. EDA (Exploratory Data Analysis)

Most of the EDA methods here are being used for all the tasks in this project, it may vary a bit, which is:

- **Data Sanity Check** [Appendix G, H]: output the images for visualization and remove the duplicate ones.
- **Checking and handling duplicates**: by hashing technique, we are able to find out the duplicated images in our data and we removed it from the dataset to ensure the data sanity.
- **Data remapping**: images were organized into folders named after their target labels (e.g., disease, variety, or age group) to streamline task-specific processing.
- **Data distribution check** [Appendix F]: by plotting out the image count of the image in each subfolders, we are started to see some imbalance problems in the dataset, which we will address in the next part.
- **Data augmenting** [Appendix F]: to address class imbalance, underrepresented classes were augmented using random transformations (rotation, zoom, brightness shift, flip). Each class was expanded to match the average number of images across all classes, improving model fairness and generalization.
- **Calculate data weights** (Tasks 1 & 2 only): For classification tasks, we computed class weights to help the model pay more attention to underrepresented classes. This method is not suitable for Task 3 (regression), where target values are continuous.
- **Data normalization**: All images were resized to 224×224 pixels and normalized to a [0,1] scale.

- **Data Splitting:** The dataset was split into 80% training and 20% validation using ImageDataGenerator for consistent preprocessing. For classification we use flow_from_directory, for regression we use flow_from_dataframe.

IV. Model Implementation

Task 1: Disease Classification:

To predict the health condition of paddy plants and identify specific diseases, we employed the following models:

- **Custom CNN:** A sequential model with three convolutional layers, batch normalization, dropout, and L2 regularization, serving as a strong baseline.
- **Simplified VGG-Style Model:** A compact adaptation of VGG using modular convolution blocks to balance depth and efficiency.
- **MobileNetV2 (from scratch):** A lightweight model using depthwise separable convolutions, trained without pretrained weights for efficient inference.

All models predict 10 disease classes and use categorical cross-entropy with label smoothing and the Adam optimizer.

Task 2: Variety Classification:

For identifying paddy varieties, we selected models capable of capturing subtle visual distinctions:

- **Custom CNN:** A sequential model with three convolutional blocks, using L2 regularization, dropout, and batch normalization to capture basic shape and texture features

EfficientNetB0 (from scratch): A compact, modern architecture with compound scaling. It was trained without pretrained weights and extended with dense layers for classification.

- **ResNet50 with Attention:** A deep residual network enhanced with an attention mechanism to focus on discriminative features. This version was also trained from scratch, with a custom classification head for fine-grained variety recognition.

All models were trained using categorical cross-entropy with label smoothing and evaluated with accuracy, precision, and recall.

Task 3: Age Prediction (Regression):

- **Custom CNN:** A sequential model with four convolutional blocks followed by dense layers. It uses dropout, batch normalization, and L2 regularization to extract growth-related visual features such as texture and leaf shape.
- **NASNetMobile (Custom Multi-Head):** A lightweight NASNet-based model trained from scratch. It uses three auxiliary regression heads (young, medium, mature) along with a final fused output to capture multi-stage age signals and improve overall prediction robustness.

Both models output continuous values representing the plant's age in days.

Training Strategy and Hyperparameter Tuning

To improve training stability and efficiency, we used both built-in and custom Keras callbacks. EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint dynamically adjusted learning rates, prevented overfitting, and saved the best-performing models. CSVLogger and TensorBoard enabled performance tracking and visualization. Two custom callbacks—TerminateNaN and LossSpikeStopper—were added to halt training on unstable loss behavior.

While hyperparameters were manually chosen, these callbacks provided adaptive control, effectively supporting training without full automated tuning.

Model Evaluation

Model performance was assessed using the following metrics:

- **Categorical Accuracy:** used for classification tasks (Task 1 and Task 2) to measure the proportion of correctly predicted labels.
- **Mean Absolute Error (MAE)**: used for the regression task (Task 3) to measure the average absolute difference between predicted and actual age values.
- **Root Mean Squared Error (RMSE):** also reported for Task 3 to provide additional insight into prediction error magnitude.

Each model's performance was compared based on these metrics to identify the most effective architecture for each task.

V. Model Training Summary [Appendix A]

Task 1: Disease Classification [Appendix B]

VGG and MobileNet achieved high validation accuracy (>93%) and low loss, outperforming the baseline CNN. Both showed fast convergence and minimal overfitting, with stable training curves. CNN lagged behind with lower accuracy and higher variance in learning.

Task 2: Variety Classification [Appendix C]

ResNet50 delivered the best overall results with strong accuracy, precision, and recall, indicating balanced and robust performance. EfficientNet showed moderate stability and improved learning over time but couldn't match ResNet's precision-recall trade-off. CNN again underperformed, showing slower convergence and higher validation loss.

Task 3: Age Prediction (Regression) [Appendix D]

The custom CNN model achieved the lowest mean absolute error (MAE) and root mean squared error (RMSE), making it the most accurate for predicting plant age.
Rasnet (NASNet-based) performed well on auxiliary outputs (young, medium, mature stages) but its final prediction head did not outperform the CNN, making it better suited for ensemble or multi-task scenarios.

VI. GUI Application for Real-World Integration [Appendix E]

To enhance usability, we developed a simple GUI using Flask (backend) and HTML/JavaScript with Dropzone (frontend). The interface allows users to upload images, which are then processed by the backend to load the appropriate trained model.

For classification tasks, it displays the predicted class with the highest confidence. For age prediction, the output is de-normalized and returned in actual day units. This demonstrates how the model can be deployed in a practical setting for real-world use.

VII. Ultimate Judgment of the Final Trained Model

Best models Across All Tasks

- **Task 1 – Disease Classification:** The VGG model is the best performer, achieving the highest validation accuracy with low loss. It showed fast convergence, stable learning, and minimal overfitting, making it reliable for classification tasks.

- **Task 2 – Variety Classification:** ResNet50 outperformed other models with strong validation accuracy, precision, and recall. Its deep architecture and skip connections enabled robust feature learning and consistent performance across epochs.

- **Task 3 – Age Prediction (Regression):** The custom CNN model delivered the lowest error in age prediction, outperforming the multi-head NASNet model. It achieved smooth convergence and strong generalization, making it most suitable for regression in this context.

Pros and Cons of the Project Execution:

What Went Well (Pros)	What Could Be Improved (Cons)
Clear task separation with modular code structure	Limited time spent on hyperparameter tuning; no automated search used
Effective use of callbacks to manage training and avoid overfitting	Bad model designed for Task 3 that can't generalized well and yield bad result on test data.
Strong model performance in classification and regression tasks	CNN models underperformed in classification compared to deeper architectures
Hands-on control by training all models locally	Local training limited scalability and training speed
Good use of EDA, augmentation, and class balancing techniques	No cross-validation; relied on single train/validation split

Future Improvements:

- Hyperparameter Optimization: Integrate automated tuning tools like Keras Tuner or Optuna.
- Learning and Figuring out different library tools for model training to catch up with the AI trend.
- Model Ensembling: Combine multiple models (e.g., CNN + NASNet) to improve robustness and accuracy.
- Improve GUI Application for easier access for everyone who need it.
- Time management would need to be enhanced to ensure the workflow is on time.
- Cross-Validation: Use k-fold validation to better assess generalization and reduce reliance on a single train/val split.

Final Judgement:

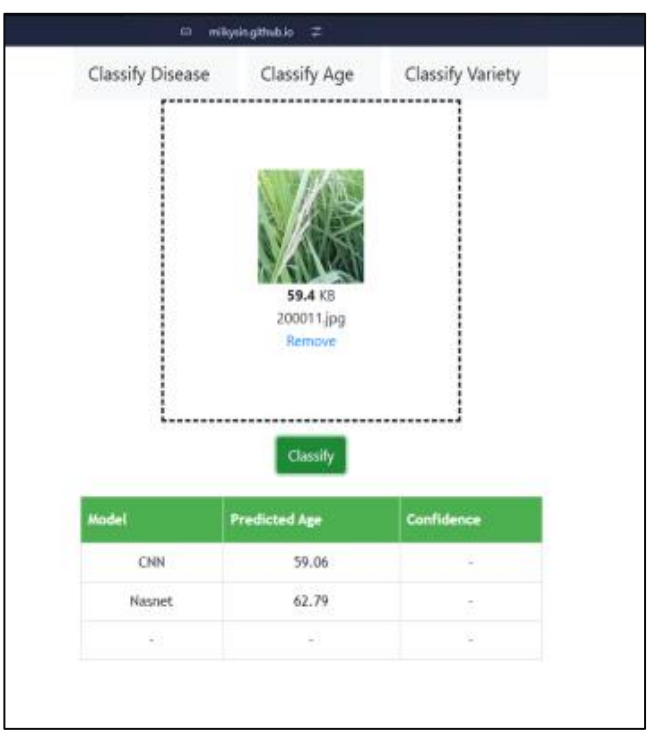
Overall, the project was well-structured and effectively addressed all three tasks—disease classification, variety classification, and age prediction—with models that performed reliably and generalized well. While deeper tuning and pretrained models could have further improved performance, the consistent use of training strategies, evaluation metrics, and modular design contributed to a solid outcome. The work demonstrates a strong foundation in machine learning workflows, with clear potential for future scaling and refinement.

VIII. Appendix

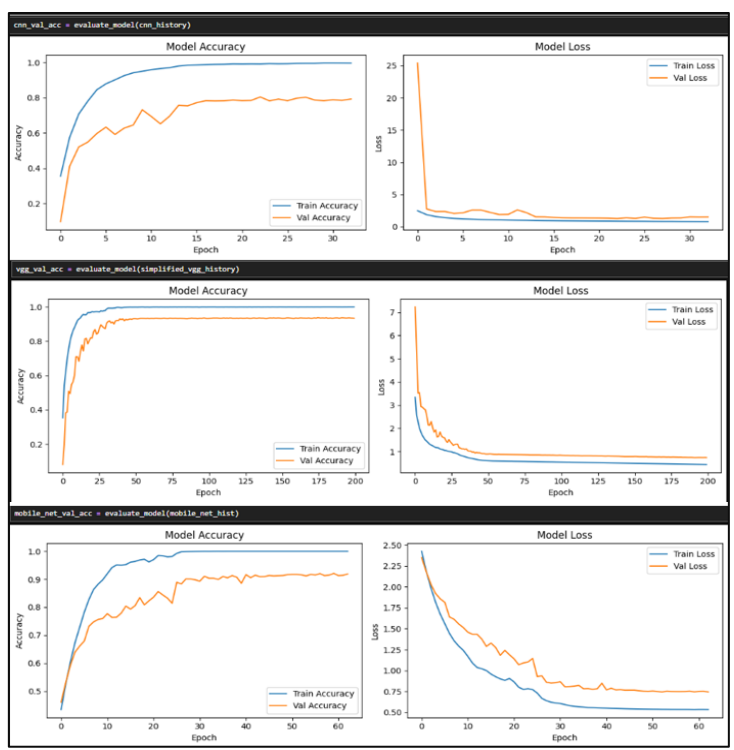
[Appendix A] Evaluation Datatable of Models

Task 1 **Disease Classification**	**Top 1. VGG**	**Accuracy:** 0.9371 Val Loss: 0.7425	Fast convergence, high accuracy, minimal overfitting
	Top 2. MobileNet	**Accuracy: 0.9299** Val Loss: 0.7442	Lightweight and close to VGG performance
	Top 3. CNN	**Accuracy: 0.8049** Val Loss: 1.2875	Slower convergence, lower overall accuracy
Task 2 **Variety Classification**	**Top 1. ResNet50**	**Accuracy: 0.9065** Precision: 0.9260 Recall: 0.8861 Loss: 0.8332	Best across all classification metrics, strong generalization
	Top 2. EfficientNet	**Accuracy: 0.7651** Precision: 0.7999 Recall: 0.7173 Loss: 1.1066	Stable learning, but underperformed on recall and final accuracy
	Top 3. CNN	**Accuracy: 0.7201** Loss: 1.4019	Lowest performance, higher variance during training
Task 3 **Age Prediction (Regression)**	**Top 1. CNN Task3**	Val Loss: 0.0433 **MAE:** 0.0943 RMSE: 0.1146	Best final prediction performance, smooth convergence
	Top 2. Rasnet (NASNet)	**Final MAE: 0.1007** Aux MAE (Young): 0.0269 Mature Loss: 0.0182	Strong on auxiliary heads but underperformed on final output

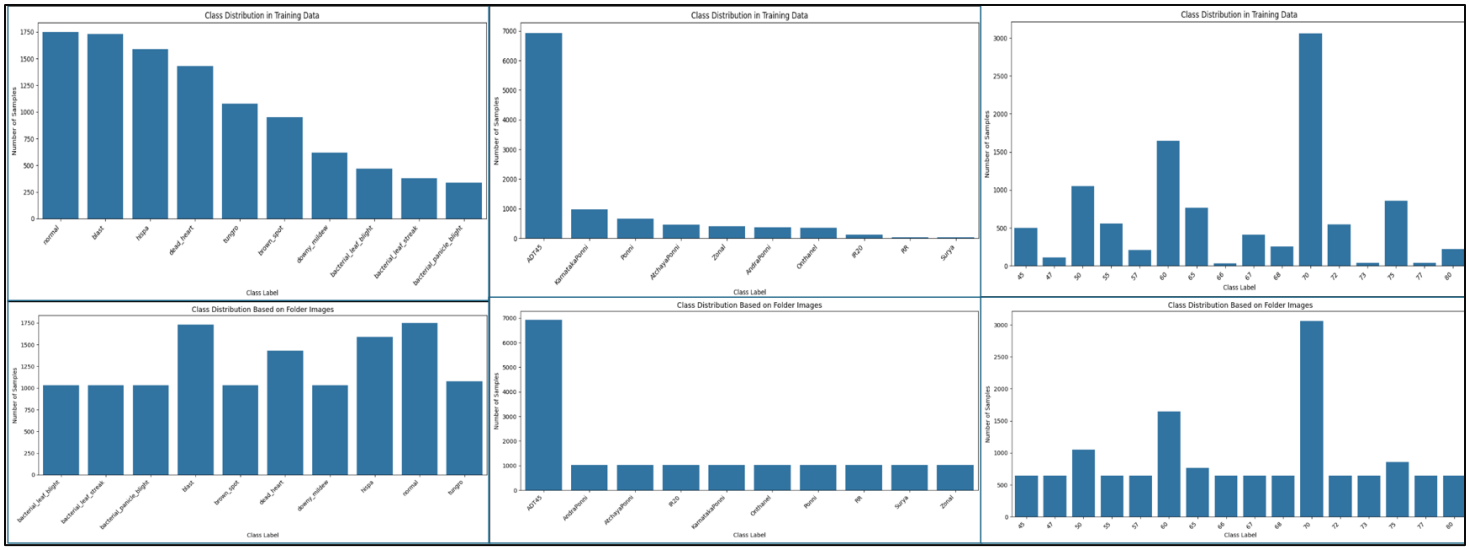
[Appendix E] Application



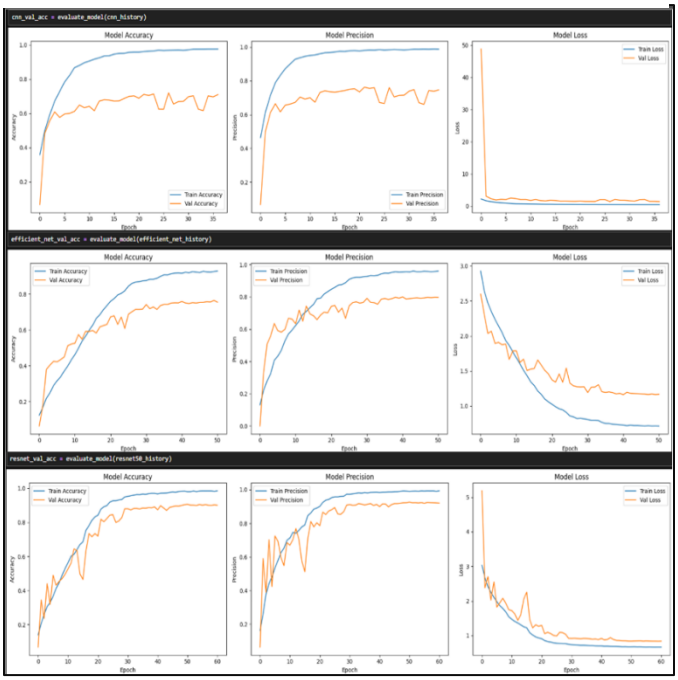
[Appendix B] Train Curve Evaluation for Task 1



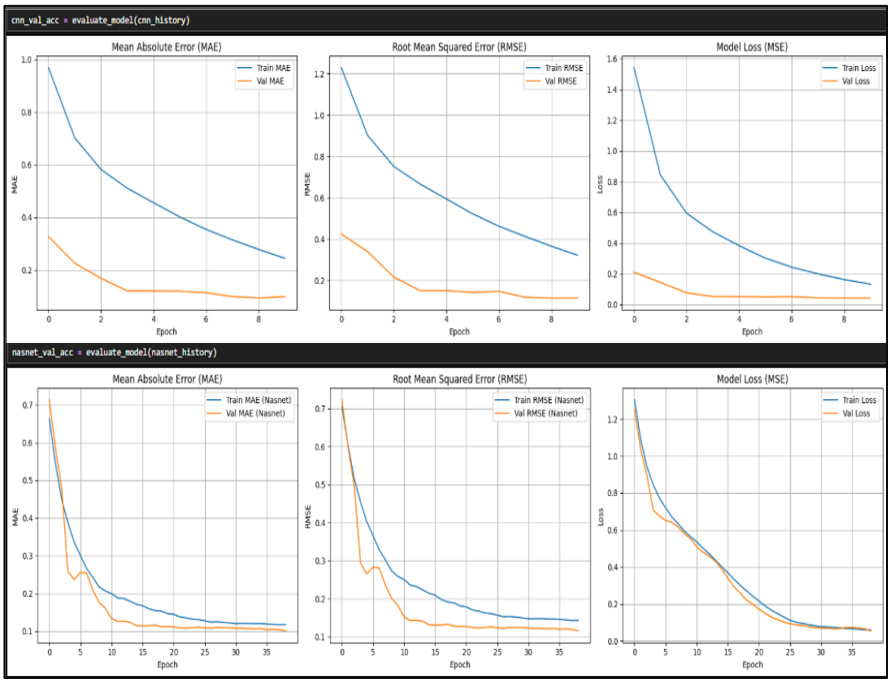
[Appendix C] Class Distribution before and after Augmentated

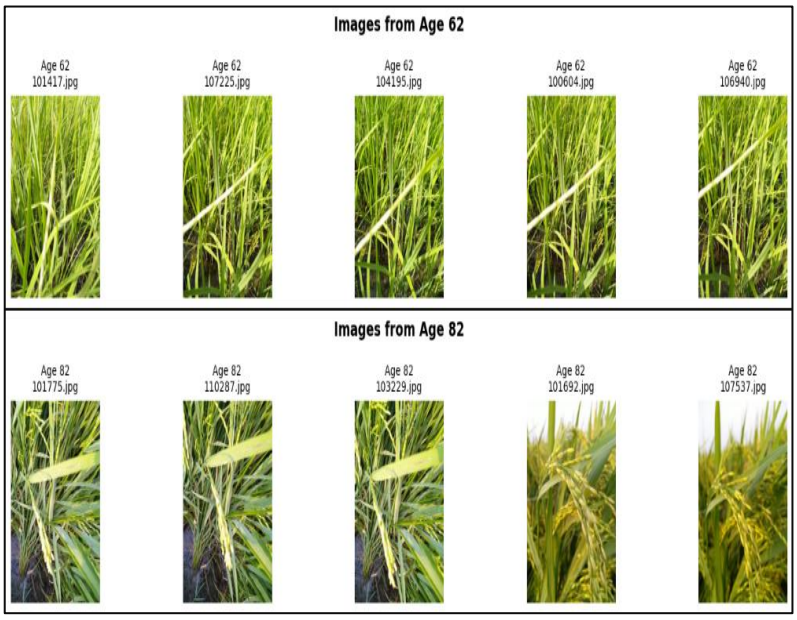


[Appendix C] Train Curve Evaluation for task 2



[Appendix D] Train Curve Evaluation for task 3



[Appendix C] Train Curve Evaluation for task 2

[Appendix C] Train Curve Evaluation for task 2